МИНИСТЕРСТВО ОБРАЗОВАНИЯ КИРОВСКОЙ ОБЛАСТИ
Кировское областное государственное профессиональное образовательное бюджетное учреждение
«Слободской колледж педагогики и социальных отношений»

ОТЧЕТ

по учебной практике

ПМ11. Разработка, администрирование и защита баз данных

Студент

Стариков Дмитрий Алексеевич

Группа 23П-2

Специальность 09.02.07 Информационные системы и программирование

Руководитель практики от колледжа:

Калинин Арсений Олегович

Руководитель практики от организации:

_____

подпись

УТВЕРЖДАЮ:

Директор

_____

Наименование организации

Слободской колледж педагогики и социальных отношений

_____/_____

подпись расшифровка

М. П.

2025-2026 уч. год

Содержание

Оглавление
- Выполнение анализа и предварительной обработки информации
- Выделение объектов и атрибутов в соответствии с заданием
- Проектирование и нормализация БД в полном соответствии с поставленной задачей и применением CASE-средств
- Выполнение построения БД в предложенной СУБД и заполнение всех таблиц с помощью соответствующих средств
- Реализация уровней доступа для различных категорий пользователей
- Создание запросов и отчетов в соответствии с заданием
- Создание и обоснование групп пользователей, принципов регистрации и системы паролей
- Выполнение резервного копирования БД и восстановление состояния БД на заданную дату
- Подготовка к защите отчета по практике.

Приложения

1. Выполнение анализа и предварительной обработки информации

На этапе анализа предметной области были выделены основные бизнес-процессы системы управления посещениями объекта КИИ. В результате составлена диаграмма вариантов использования (Use Case Diagram) в нотации UML, отображающая функциональные возможности для четырёх категорий пользователей: посетитель, сотрудник подразделения, сотрудник общего отдела и сотрудник охраны.

Рисунок 1 – UML-диаграмма

Акторы: Посетитель, Сотрудник подразделения, Сотрудник общего отдела, Сотрудник охраны.

Варианты использования:

Посетитель: Зарегистрироваться, Авторизоваться, Подать индивидуальную заявку, Подать групповую заявку, Просмотреть свои заявки, Добавить посетителя в чёрный список (последнее доступно также сотрудникам подразделения).

Сотрудник подразделения: Авторизоваться, Просмотреть заявки своего подразделения, Отфильтровать заявки, Зафиксировать прибытие в подразделение, Зафиксировать убытие из подразделения.

Сотрудник общего отдела: Авторизоваться, Просмотреть все заявки, Одобрить заявку, Отклонить заявку, Отправить уведомление по email (включается в одобрение/отклонение).

Сотрудник охраны: Авторизоваться, Просмотреть одобренные заявки на сегодня, Осуществить поиск по ФИО/паспорту, Зафиксировать вход посетителя, Зафиксировать выход посетителя.

Связи между акторами и вариантами использования отражают ролевую модель доступа.

Для представления структуры данных была разработана ER-диаграмма (сущность-связь), отражающая основные сущности и их взаимосвязи.

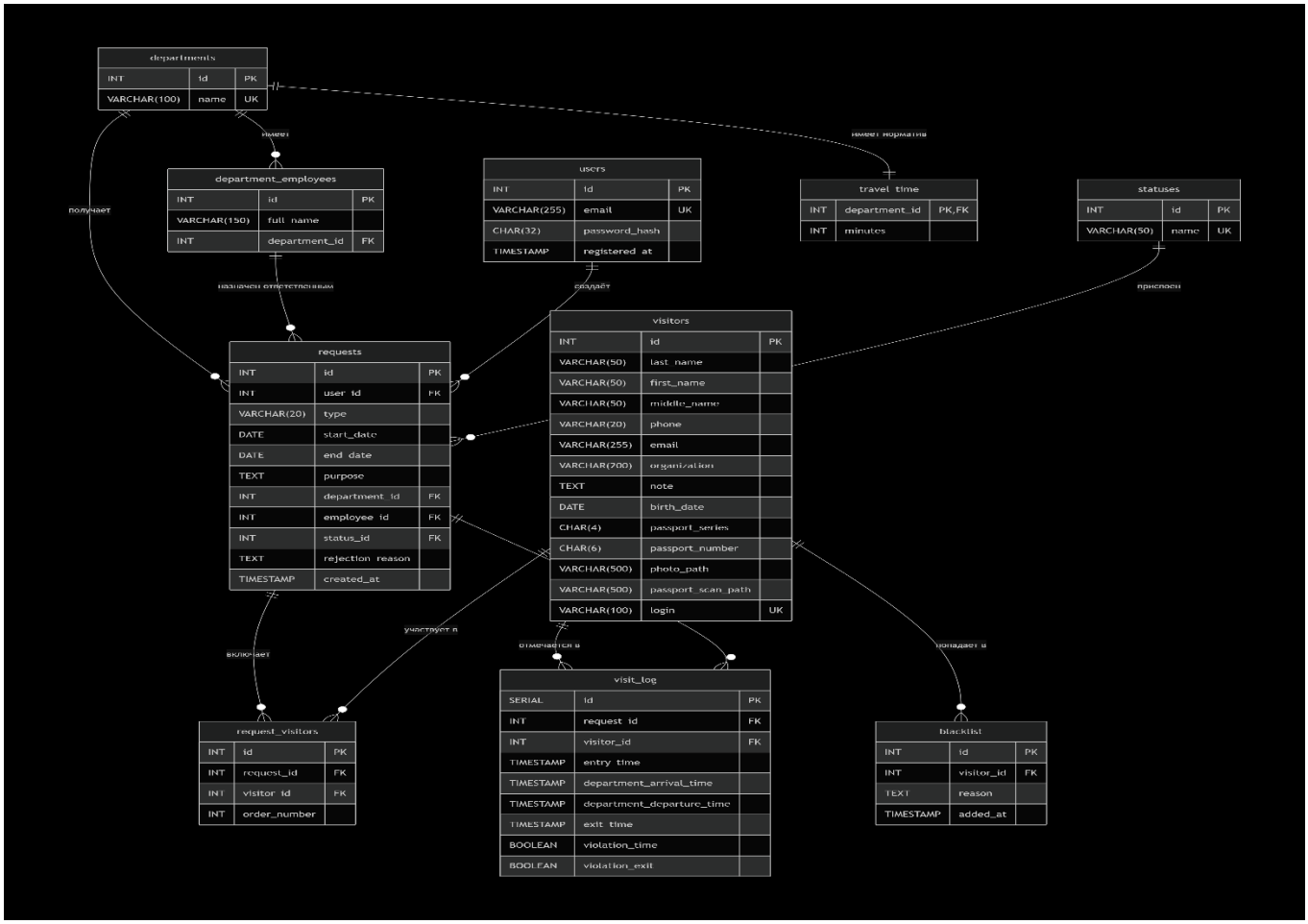

Рисунок 2 – ER-диаграмма базы данных

2. Выделение объектов и атрибутов в соответствии с заданием

В ходе проектирования был составлен детальный словарь данных (Data Dictionary), содержащий описание всех таблиц базы данных, их атрибутов, типов данных, ограничений и назначения.

Таблица 1 – Фрагмент словаря данных

Таблица	Поле	Тип	Ограничения	Описание
visitors	last_name	VARCHAR(50)	NOT NULL	Фамилия посетителя
visitors	passport_series	CHAR(4)	CHECK (4 цифры)	Серия паспорта
requests	type	VARCHAR(20)	CHECK (personal/group)	Тип заявки

3. Проектирование и нормализация БД в полном соответствии с поставленной задачей и применением CASE-средств

Проектирование базы данных выполнено с использованием принципов нормализации. Все таблицы приведены к третьей нормальной форме (3НФ), что обеспечивает отсутствие избыточности данных и аномалий обновления.

1НФ – все атрибуты атомарны, отсутствуют повторяющиеся группы.

2НФ – каждый неключевой атрибут полностью зависит от первичного ключа.

3НФ – отсутствуют транзитивные зависимости между неключевыми атрибутами.

Для визуализации структуры базы данных была построена физическая схема БД, отображающая таблицы, поля, первичные и внешние ключи.

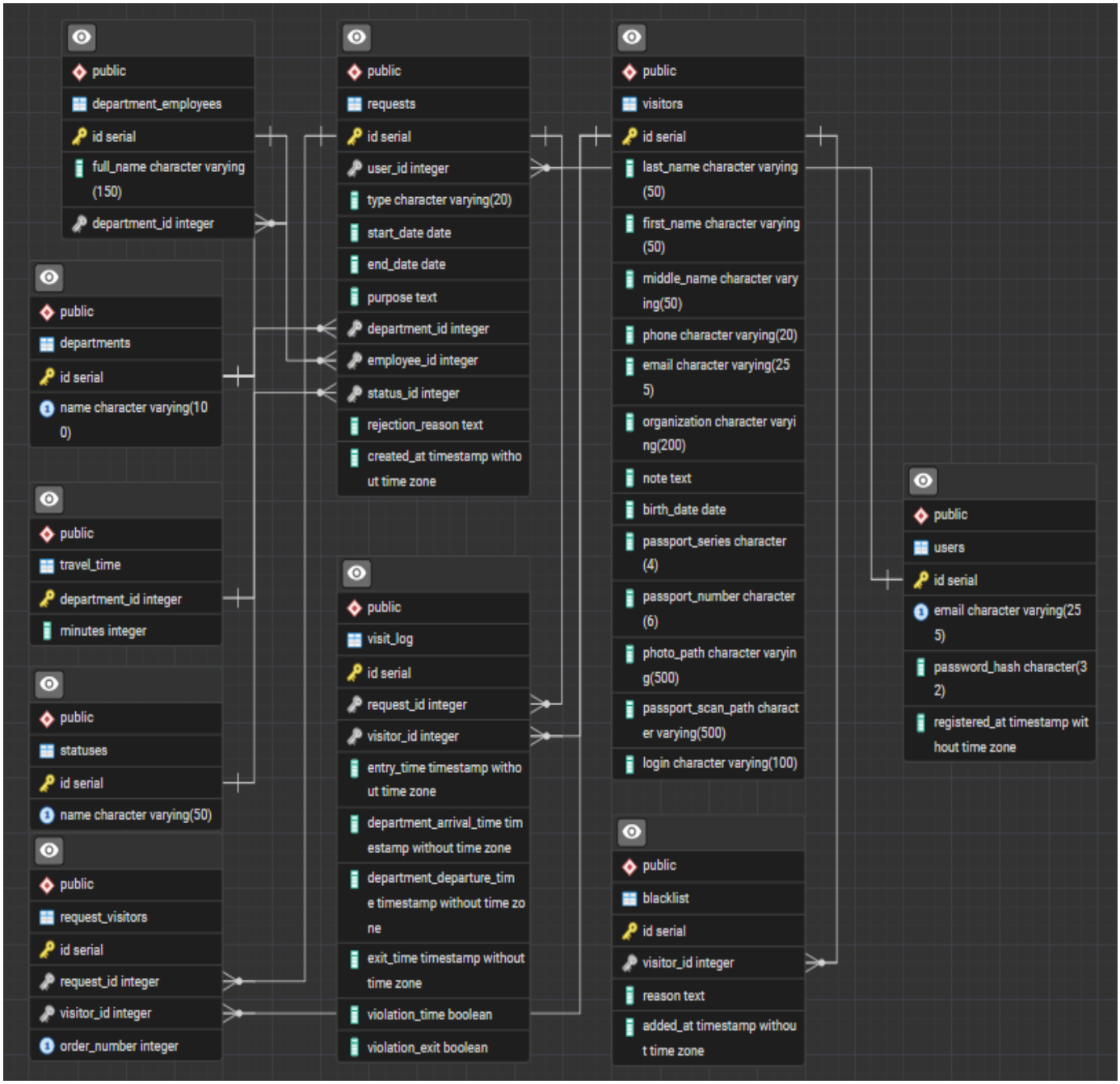

Рисунок 3 – Физическая схема базы данных

4. Выполнение построения БД в предложенной СУБД и заполнение всех таблиц с помощью соответствующих средств

База данных реализована в СУБД PostgreSQL 18.3. Создание структуры выполнено с помощью SQL-скрипта, включающего:

Создание таблиц с ограничениями PRIMARY KEY, FOREIGN KEY, CHECK.

Создание последовательностей для автоинкрементных полей.

Создание представления ViewListRequests.

Создание хранимых функций filteringrequests, sp_login_user, sp_register_user, generate_visitor_login.

Создание триггера для автоматической генерации логина посетителя.

Заполнение таблиц тестовыми данными осуществлено через команды INSERT и COPY. Тестовый набор включает:

3 статуса заявок,

7 подразделений,

7 сотрудников подразделений,

30 пользователей (зарегистрированных посетителей),

30 записей о посетителях,

17 заявок (из них 15 личных и 2 групповых),

30 связей заявок с посетителями,

7 записей о времени перемещения до подразделений

	id [PK] integer	user_id integer	type character varying (20)	start_date date	end_date date	purpose text	department_id integer	employee_id integer	status_id integer	rejection_reason text
1	17	23	group	2023-04-24	2023-04-25	Экскурсия на производство (вторая груп...	1	1	2	[null]
2	16	16	group	2023-04-24	2023-04-25	Экскурсия на производство	1	1	2	[null]
3	15	15	personal	2023-04-28	2023-04-29	Личное посещение	3	3	2	[null]
4	14	14	personal	2023-04-28	2023-04-29	Личное посещение	2	2	2	[null]
5	13	13	personal	2023-04-28	2023-04-29	Личное посещение	1	1	2	[null]
6	12	12	personal	2023-04-27	2023-04-28	Личное посещение	3	3	2	[null]
7	11	11	personal	2023-04-27	2023-04-28	Личное посещение	2	2	2	[null]
8	10	10	personal	2023-04-27	2023-04-28	Личное посещение	1	1	2	[null]
9	9	9	personal	2023-04-26	2023-04-27	Личное посещение	3	3	2	[null]
10	8	8	personal	2023-04-26	2023-04-27	Личное посещение	2	2	2	[null]
11	7	7	personal	2023-04-26	2023-04-27	Личное посещение	1	1	2	[null]
12	6	6	personal	2023-04-25	2023-04-26	Личное посещение	3	3	2	[null]
13	5	5	personal	2023-04-25	2023-04-26	Личное посещение	2	2	2	[null]
14	4	4	personal	2023-04-25	2023-04-26	Личное посещение	1	1	2	[null]
15	3	3	personal	2023-04-24	2023-04-25	Личное посещение	3	3	2	[null]
16	2	2	personal	2023-04-24	2023-04-25	Личное посещение	2	2	2	[null]
17	1	1	personal	2023-04-24	2023-04-25	Личное посещение	1	1	2	[null]

Рисунок 4 – Содержимое таблицы requests в pgAdmin

5. Реализация уровней доступа для различных категорий пользователей

В приложении реализовано четыре роли пользователей с разграничением функциональных возможностей как на уровне клиентского приложения, так и через логику запросов к БД.

5.1. Посетитель

Может зарегистрироваться (создать учётную запись в таблице users).

Авторизуется по email и паролю.

Подаёт индивидуальные и групповые заявки (создание записей в requests и request_visitors).

Просматривает список своих заявок через представление ViewListRequests с фильтром по user_email.

5.2. Сотрудник подразделения

Авторизуется по коду сотрудника (ID в таблице department_employees).

При входе проверяется принадлежность к подразделению (кроме общего отдела и охраны – ID 6 и 7).

Видит только одобренные заявки своего подразделения.

Может фиксировать время прибытия и убытия посетителей в visit_log.

Имеет возможность добавить посетителя в чёрный список через диалоговое окно.

5.3. Сотрудник общего отдела

Авторизуется по коду сотрудника, привязанному к подразделению с ID = 6.

Имеет доступ ко всем заявкам через функцию FilteringRequests.

Может одобрять или отклонять заявки, меняя статус и отправляя email-уведомления посетителям (через EmailService).

При проверке заявки автоматически выявляет наличие посетителей в чёрном списке и отклоняет заявку.

5.4. Сотрудник охраны

Авторизуется по коду сотрудника подразделения с ID = 7.

Видит только одобренные заявки на текущую дату (или отфильтрованные по дате).

Может осуществлять поиск по ФИО или паспортным данным.

Фиксирует время входа и выхода посетителей через турникет (звуковой сигнал и запись в visit_log).

Доступ к данным регулируется на уровне SQL-запросов, где в зависимости от роли подставляются соответствующие условия WHERE.

6. Создание запросов и отчетов в соответствии с заданием

Для формирования отчетов и фильтрации данных используются объекты базы данных: представления, функции, хранимые процедуры.

6.1. Представление ViewListRequests

Объединяет данные из таблиц requests, departments, department_employees, statuses, users, request_visitors, visitors и формирует удобный для отображения список заявок с агрегированным списком посетителей через запятую.

sql

```
CREATE VIEW public.viewlistrequests AS
SELECT r.id AS request_id,
    r.type,
    r.start_date,
    r.end_date,
    r.purpose,
    r.created_at,
    d.name AS department_name,
    de.full_name AS employee_full_name,
    s.name AS status_name,
    u.email AS user_email,
    string_agg((((((v.last_name)::text || ' '::text) || (v.first_name)::text) || ' '::text) || (COALESCE(v.middle_name, ''::character varying))::text), ', '::text ORDER BY rv.order_number) AS visitors_list
FROM ((((((public.requests r
    JOIN public.departments d ON ((r.department_id = d.id)))
    JOIN public.department_employees de ON ((r.employee_id = de.id)))
    JOIN public.statuses s ON ((r.status_id = s.id)))
    JOIN public.users u ON ((r.user_id = u.id)))
    LEFT JOIN public.request_visitors rv ON ((r.id = rv.request_id)))
    LEFT JOIN public.visitors v ON ((rv.visitor_id = v.id)))
```

```
GROUP BY r.id, d.name, de.full_name, s.name, u.email;
```

Рисунок 5 - Код представления ViewListRequests

6.2. Функция фильтрации FilteringRequests

Позволяет сотрудникам общего отдела и охраны фильтровать заявки по типу, подразделению и статусу.

sql

```
CREATE FUNCTION public.filteringrequests(p_type VARCHAR DEFAULT NULL, p_department_id INT DEFAULT NULL, p_status_id INT DEFAULT NULL)
RETURNS SETOF public.viewlistrequests AS $$
BEGIN
  RETURN QUERY
  SELECT *
  FROM public.ViewListRequests v
  WHERE (p_type IS NULL OR v.type = p_type)
    AND (p_department_id IS NULL OR v.department_name = (SELECT name FROM public.departments WHERE id = p_department_id))
    AND (p_status_id IS NULL OR v.status_name = (SELECT name FROM public.statuses WHERE id = p_status_id));
END;
$$ LANGUAGE plpgsql;
```

Рисунок 6 – Код функции FilteringRequests

6.3. Хранимые процедуры аутентификации и регистрации

sp_login_user(p_email VARCHAR, p_password_hash CHAR) – возвращает TRUE, если пользователь с указанным email и хешем пароля существует.

sp_register_user(p_email VARCHAR, p_password_hash CHAR) – регистрирует нового пользователя, если email не занят.

```
CREATE FUNCTION public.sp_login_user(p_email VARCHAR, p_password_hash CHAR)
RETURNS BOOLEAN AS $$
```

```
BEGIN
  RETURN EXISTS (SELECT 1 FROM users WHERE email = p_email AND password_hash = p_password_hash);
END;
$$ LANGUAGE plpgsql;

CREATE FUNCTION public.sp_register_user(p_email VARCHAR, p_password_hash CHAR)
RETURNS BOOLEAN AS $$
BEGIN
  IF EXISTS (SELECT 1 FROM users WHERE email = p_email) THEN
    RETURN FALSE;
  END IF;
  INSERT INTO users (email, password_hash, registered_at) VALUES (p_email, p_password_hash, NOW());
  RETURN TRUE;
END;
$$ LANGUAGE plpgsql;
```

Рисунок 7 – Код хранимых процедур

6.4. Триггер генерации логина посетителя

Автоматически формирует логин посетителя при добавлении новой записи в формате префикс_email_id.

sql

```
CREATE FUNCTION public.generate_visitor_login() RETURNS TRIGGER AS $$
DECLARE
  email_prefix TEXT;
  new_login TEXT;
BEGIN
  email_prefix := SPLIT_PART(NEW.email, '@', 1);
  new_login := CONCAT(email_prefix, '_', NEW.id);
  UPDATE public.visitors SET login = new_login WHERE id = NEW.id;
```

```
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER visitor_generate_login_trigger
AFTER INSERT ON public.visitors
FOR EACH ROW EXECUTE FUNCTION public.generate_visitor_login();
```

Рисунок 8 – Код процедур аутентификации

7. Разработка модулей программы

Клиентское приложение разработано на платформе .NET Framework 4.8 с использованием WPF (Windows Presentation Foundation) и языка C#. Архитектура приложения построена по принципу разделения на слои:

Data Layer – класс DatabaseHelper с методами ExecuteScalar, ExecuteNonQuery, ExecuteQuery для работы с PostgreSQL через Npgsql.

Models – классы-сущности: Visitor, Department, DepartmentEmployee, Status, RequestViewItem.

Views – XAML-окна и страницы для каждой роли.

Services – EmailService для отправки уведомлений через SMTP.

Основные окна приложения:

Таблица 2 – Состав модулей пользовательского интерфейса

Окно / Страница	Назначение
MainWindow	Стартовое окно выбора режима входа (посетитель, общий отдел, охрана, подразделение)
VisitorLoginWindow	Авторизация посетителя по email и паролю
VisitorRegisterWindow	Регистрация нового посетителя
VisitorMainWindow	Главное окно посетителя с навигацией по заявкам
IndividualRequestWindow	Форма подачи индивидуальной заявки
GroupRequestWindow	Форма подачи групповой заявки с возможностью добавления нескольких посетителей
MyRequestsPage	Страница просмотра заявок текущего пользователя
GeneralDepartmentLogin	Вход для сотрудника общего отдела

Window	
GeneralDepartmentMainWindow	Главное окно общего отдела с фильтрацией и списком всех заявок
RequestReviewWindow	Окно формальной проверки заявки с возможностью одобрения/отклонения
SecurityLoginWindow	Вход для сотрудника охраны
SecurityMainWindow	Главное окно охраны со списком одобренных заявок и поиском
SecurityVisitWindow	Управление входом/выходом посетителей (фиксация времени)
DepartmentLoginWindow	Вход для сотрудника подразделения
DepartmentMainWindow	Главное окно подразделения с одобренными заявками
DepartmentVisitWindow	Фиксация прибытия/убытия посетителей в подразделении
BlacklistReasonDialog	Диалог ввода причины добавления в чёрный список
VisitorEditDialog	Диалог добавления/редактирования посетителя

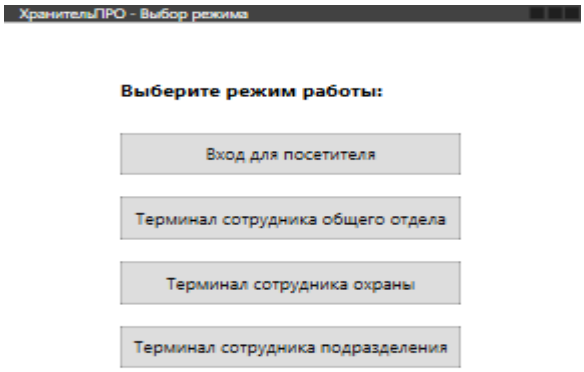

Рисунок 9 – Скриншот главного окна выбора режима

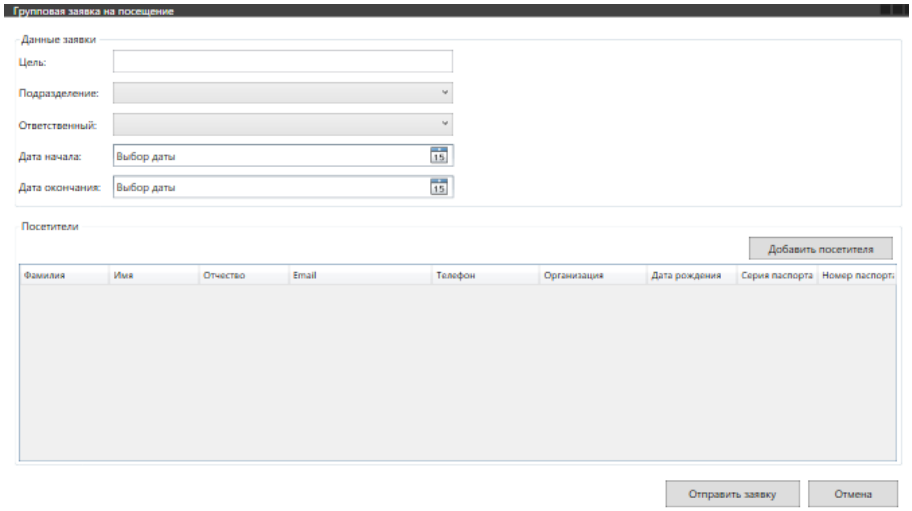

Рисунок 10 – Скриншот окна подачи групповой заявки

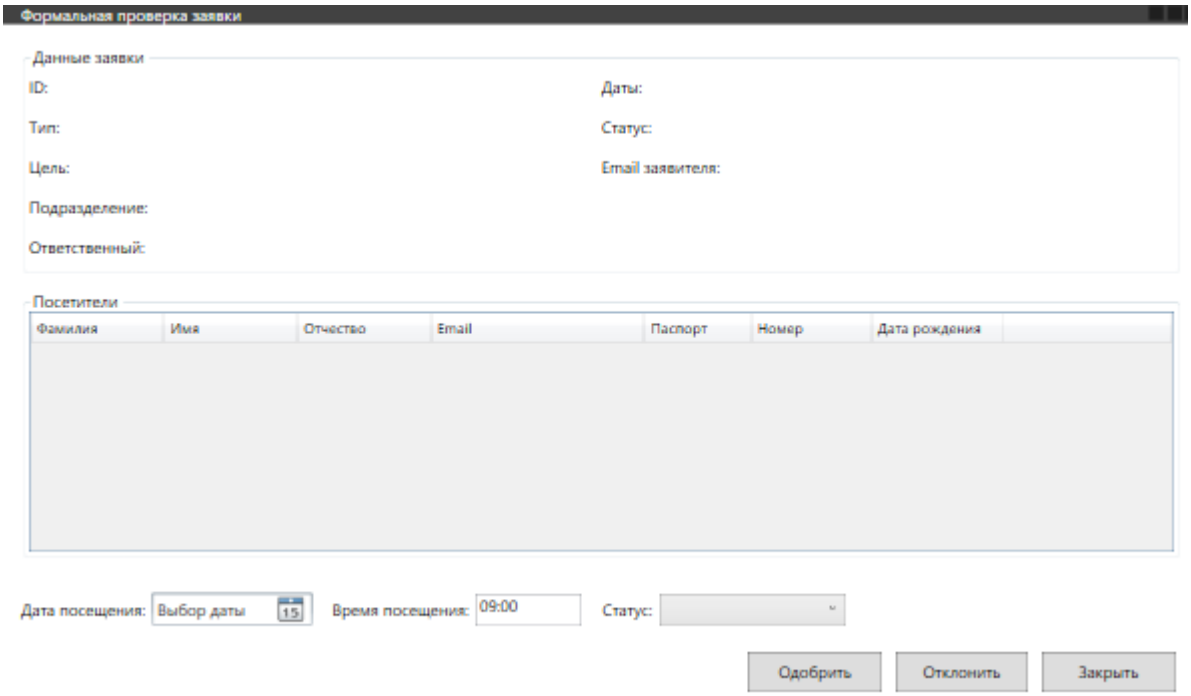

Рисунок 11 – Скриншот окна проверки заявки сотрудником общего отдела

8. Создание и обоснование групп пользователей, принципов регистрации и системы паролей

Выделение четырёх групп пользователей обусловлено различием функциональных обязанностей:

Посетители – внешние пользователи, желающие получить доступ на объект. Регистрируются самостоятельно, указывая email и пароль. Пароль хранится в виде MD5-хеша (в промышленной версии рекомендуется использовать более стойкий алгоритм, например, bcrypt).

Сотрудники подразделений – принимающая сторона, ответственные за сопровождение посетителей. Авторизуются по уникальному коду сотрудника (ID), который выдается администратором системы.

Сотрудники общего отдела – выполняют проверку заявок и принимают решение о допуске. Имеют расширенные права на изменение статусов заявок.

Сотрудники охраны – контролируют физический доступ на объект. Видят только актуальные одобренные заявки.

Принцип регистрации посетителей:

Проверка уникальности email.

Хеширование пароля.

Создание записи в таблице users.

При последующем создании заявок посетитель указывает свои паспортные данные, которые сохраняются в таблице visitors.

Для сотрудников организации учётные записи не создаются через регистрацию – их данные предварительно заносятся в таблицы departments и department_employees администратором.

9. Выполнение резервного копирования БД и восстановление состояния БД на заданную дату

Резервное копирование базы данных PostgreSQL выполняется утилитой pg_dump. Пример команды для создания полного дампа:

bash

```
pg_dump -U postgres -h localhost -F c -b -v -f "C:\backup\hranitelpro_20260420.backup" hranitelpro
```

Рисунок 12 – Команда создания дампа

Для восстановления базы данных из резервной копии используется утилита pg_restore:

bash

```
pg_restore -U postgres -h localhost -d hranitelpro -v "C:\backup\hranitelpro_20260420.backup"
```

Рисунок 13 – Команда восстановления БД

Также в приложении может быть реализована функция создания резервной копии по расписанию с использованием планировщика задач ОС.

Предоставленный SQL-скрипт (Postgres Script.sql) также является текстовым представлением структуры и данных, которое можно использовать для восстановления БД на любом сервере PostgreSQL.

```
C:\Users\PC33-2>cd "C:\Program Files\PostgreSQL\18\bin"

C:\Program Files\PostgreSQL\18\bin>pg_dump -U postgres -h localhost -F c -b -v -f "C:\backup\hranitelpro.backup" hranitelpro
Пароль:
pg_dump: выполняется SELECT pg_catalog.set_config('search_path', '', false);
pg_dump: последний системный OID: 16383
pg_dump: чтение расширений
pg_dump: выявление членов расширений
pg_dump: чтение схем
pg_dump: чтение пользовательских таблиц
pg_dump: чтение пользовательских функций
pg_dump: чтение пользовательских типов
pg_dump: чтение процедурных языков
pg_dump: чтение пользовательских агрегатных функций
pg_dump: чтение пользовательских операторов
pg_dump: чтение пользовательских методов доступа
pg_dump: чтение пользовательских классов операторов
pg_dump: чтение пользовательских семейств операторов
pg_dump: чтение пользовательских анализаторов текстового поиска
pg_dump: чтение пользовательских шаблонов текстового поиска
pg_dump: чтение пользовательских словарей текстового поиска
pg_dump: чтение пользовательских конфигураций текстового поиска
pg_dump: чтение пользовательских оболочек сторонних данных
pg_dump: чтение пользовательских сторонних серверов
pg_dump: чтение прав по умолчанию
pg_dump: чтение пользовательских правил сортировки
pg_dump: чтение пользовательских преобразований
pg_dump: чтение приведений типов
pg_dump: чтение преобразований
pg_dump: чтение информации о наследовании таблиц
pg_dump: чтение событийных триггеров
pg_dump: поиск таблиц расширений
pg_dump: поиск связей наследования
pg_dump: чтение информации о столбцах интересующих таблиц
pg_dump: поиск выражений по умолчанию для таблиц
pg_dump: поиск ограничений-проверок для таблиц
pg_dump: пометка наследованных столбцов в подтаблицах
pg_dump: чтение информации о секционировании
pg_dump: чтение индексов
pg_dump: пометка индексов в секционированных таблицах
pg_dump: чтение расширенной статистики
pg_dump: чтение ограничений
pg_dump: чтение триггеров
pg_dump: чтение правил перезаписи
pg_dump: чтение политик
pg_dump: чтение политик защиты на уровне строк
pg_dump: чтение публикаций
pg_dump: чтение информации о таблицах, включённых в публикации
pg_dump: чтение информации о схемах, включённых в публикации
```

Рисунок 14 – Выполнение резервного копирования базы данных

Заключение

В ходе учебной практики была разработана информационная система «ХранительПРО», предназначенная для автоматизации процессов подачи и обработки заявок на посещение объекта критической информационной инфраструктуры.

Реализованы следующие компоненты:

База данных под управлением PostgreSQL, состоящая из 10 взаимосвязанных таблиц, представлений, хранимых функций и триггеров.

Клиентское WPF-приложение с разграничением доступа для четырёх категорий пользователей.

Функции регистрации и авторизации, подачи индивидуальных и групповых заявок, формальной проверки заявок, фиксации времени прохода и перемещения посетителей.

Механизмы уведомления посетителей по электронной почте.

Возможность ведения чёрного списка.

Полученные навыки включают проектирование реляционных баз данных, разработку многослойных приложений на C# с использованием WPF, работу с СУБД PostgreSQL, реализацию бизнес-логики на стороне сервера (хранимые процедуры, триггеры).

Система готова к внедрению и может быть доработана в части усиления криптографической защиты паролей, добавления двухфакторной аутентификации и интеграции с оборудованием турникетов.

Приложение

Приложение 1

Ссылка на Github

Ссылка на github - https://github.com/Grozvvhy/Praktika

QR-Code